Đồ thị, BFS và DFS

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 5: Ưu tiên và đồ thị.
- Các bài trong chương: Heap và hàng đợi ưu tiên; Đồ thị, BFS và DFS.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Biểu diễn đồ thị và chọn chiến lược duyệt đúng mục tiêu.

Kiến thức nền

Đồ thị gồm đỉnh và cạnh, có thể có hướng hoặc vô hướng. Danh sách kề lưu các láng giềng của mỗi đỉnh; ma trận kề dùng ô biểu diễn quan hệ từng cặp đỉnh. Với đồ thị thưa, danh sách kề thường tiết kiệm bộ nhớ hơn ma trận $O(V^2)$.

BFS dùng queue để duyệt theo lớp. Trong đồ thị không trọng số, BFS tìm số cạnh ít nhất từ đỉnh xuất phát tới các đỉnh đến được. DFS đi sâu theo nhánh, có thể dùng đệ quy hoặc stack. Cả hai cần quản lý đã thăm để tránh lặp với chu trình.

Với danh sách kề và cách triển khai chuẩn, duyệt toàn phần liên quan có chi phí $O(V+E)$. Một lần duyệt từ s không bảo đảm thăm hết đồ thị không liên thông. Để duyệt toàn bộ, có thể bắt đầu lại ở đỉnh chưa thăm. BFS thông thường không tự giải đường đi ngắn nhất theo trọng số tùy ý.

1. Đọc đồ thị từ dữ liệu

Đồ thị có hướng cần ghi cạnh theo chiều; vô hướng thường thêm quan hệ hai chiều vào danh sách kề. Số đỉnh V và cạnh E cần nhất quán với cách lưu. Khi có self-loop hoặc cạnh trùng, quy định có giữ hay loại thay vì đếm tùy ý.

2. BFS trả khoảng cách ít cạnh nhất

```
from collections import deque

def bfs_distances(graph, start):
    distance = {start: 0}
    queue = deque([start])
    while queue:
```

```
        node = queue.popleft()
        for nxt in graph.get(node, []):
            if nxt not in distance:
                distance[nxt] = distance[node] + 1
                queue.append(nxt)
    return distance

g = {'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': []}
assert bfs_distances(g, 'A')['D'] == 2
```

Đánh dấu khi phát hiện giúp một đỉnh không bị thêm lặp nhiều lần. Hàm chỉ trả các đỉnh đến được từ start.

3. DFS và lựa chọn chiến lược

DFS có thể ghi thời điểm vào/ra để hỗ trợ nhiều bài toán, nhưng một lần DFS không tự cho đường ít cạnh nhất. Đệ quy có thể vượt độ sâu khi đồ thị lớn; stack tường minh là một lựa chọn. Nếu cần đường theo trọng số, phải dùng thuật toán phù hợp điều kiện trọng số, không áp BFS thường cho mọi trường hợp.

4. Bài tập và đáp án

Thêm E cô lập: BFS từ A không có E trong kết quả. Thêm D→A tạo chu trình: hàm vẫn kết thúc vì đã có distance. Đổi thứ tự kề có thể đổi thứ tự khám phá nhưng không đổi khoảng cách ít cạnh nhất. Nộp bảng queue qua từng bước và đường đi lấy qua mảng cha được bổ sung.

> **Bài tập vận dụng bổ sung**
>
> Viết danh sách kề, vẽ vết queue BFS và stack DFS. Thêm đỉnh E cô lập rồi kiểm tra một lần BFS từ A có thăm E không.

Hướng giải bài tập bổ sung

Không thăm E nếu không có đường nối. Đánh dấu phát hiện phù hợp để tránh đưa cùng đỉnh vào queue lặp lại nhiều lần.

Câu hỏi tự kiểm tra

1. BFS thường dùng cấu trúc nào?
2. DFS có thể triển khai bằng gì?
3. BFS tìm đường ít cạnh nhất trong loại đồ thị nào?
4. Vì sao cần tập đã thăm?
5. Duyệt bằng danh sách kề thường có chi phí nào?

Gợi ý đáp án và điểm cần nhớ

- Queue. Queue tạo thứ tự theo lớp.
- Stack hoặc đệ quy. Ngăn xếp hỗ trợ quay lại sau nhánh sâu.
- Đồ thị không trọng số. Mỗi lớp BFS tương ứng thêm một cạnh.
- Tránh lặp lại do chu trình. Quản lý đã thăm giúp duyệt hữu hạn và hiệu quả.
- O(V+E). Xét đỉnh và cạnh một số lần hữu hạn.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html